Developing AI-Powered Conversational Applications Using ChatGPT

Table of Contents

Introduction 3
Appendix A. Internationalization..... **Error! Bookmark not defined.**

Introduction

Artificial intelligence is transforming how software applications are designed, developed, and used. Traditional applications require users to navigate menus, complete forms, and perform predefined operations. AI-powered conversational applications allow users to interact with software using natural language, making applications more flexible, intuitive, and intelligent.

This book explores that transformation through a practical, hands-on approach, demonstrating how to use ChatGPT and Codex to develop AI-powered conversational applications that combine traditional software functionality with natural-language interaction, relational databases, and semantic search.

The Applications We Will Develop

In Part One, we will use ChatGPT to develop a small conversational database application called Phone Book as a proof of concept. The application will contain a single database table and demonstrate how users can interact with data through natural-language conversations.

In Part Two, readers will use Codex to develop a more sophisticated AI-powered Personal Assistant for Android and desktop computers running Windows, Linux, or macOS. In addition to traditional personal assistant functions, such as managing contacts, appointments, reminders, and tasks, the application will support advanced capabilities, including:

Conversational database operations: Creating, retrieving, modifying, and analyzing records through natural-language instructions.

- Intelligent document management: Building and organizing a library of documents, extracting information, and improving document content using AI.
- Semantic search: Finding information based on meaning rather than exact words or phrases.
- Personalized knowledge: Accumulating information about the user's activities, business, terminology, and preferences to provide increasingly relevant assistance.
- Cross-platform synchronization: Maintaining independent local databases and document libraries on Android and Windows, Linux or Mac with information synchronized between them.

For example, users will be able to ask:

- Show me all expenses from last month.
- Find the document where we discussed the equipment problems.
- Summarize this report and improve its recommendations.
- Remind me to contact John next Thursday.

The application will interpret these requests and perform the appropriate operations, asking for clarification or confirmation when necessary.

AI as a Development Tool

In its first role, AI acts as a programming assistant, helping developers throughout the software development lifecycle. Using ChatGPT and Codex, we will demonstrate how AI can assist with defining requirements, designing application architecture, creating database structures, generating source code, debugging errors, testing software, and preparing documentation.

Rather than writing every line of code manually, developers can describe their objectives in natural language and collaborate with AI to develop solutions. This approach, sometimes called *Prompt-driven development*, allows programmers to concentrate more on application design, functionality, and problem-solving while delegating many routine programming tasks to AI.

However, AI does not eliminate the need for programming knowledge. Developers must understand the generated code, evaluate proposed solutions, verify results, and maintain control over the development process. AI is a development assistant, not a replacement for the programmer.

Readers will learn how large language models interact with application code and databases, while maintaining the reliability, security, and data integrity required of professional software.

Android and Windows, Linux or Mac Development

This book will guide readers through the design and development of corresponding Android and desktop applications for Windows, Linux, or macOS, all providing equivalent functionality. Each application will maintain its own local relational database and document library, with synchronization across platforms to ensure consistent access to information. Both applications will incorporate AI-powered conversational interfaces, allowing users to manage structured data, search documents, and perform everyday tasks using natural language.

Learning Objectives

After studying this book, readers should be able to design, develop, and deploy AI-powered applications that combine conversational interfaces, relational databases, and semantic document search.

Specifically, readers will acquire practical skills in AI-assisted software development using ChatGPT and Codex, Android application development with Android Studio and Kotlin, and Windows, Linux or Mac application development using Qt.

Prerequisites

This book is intended for application programmers familiar with at least one programming language, such as C++ or Java, and with basic knowledge of SQL and relational databases. Familiarity with Windows, Linux, or macOS command-line tools, as well as Kotlin, Python, and JSON, is helpful but not required. ChatGPT and Codex will assist readers in learning unfamiliar tools and technologies throughout the development process.

No previous experience with artificial intelligence or Android and cross-platform application development is required.

Our Approach

This book emphasizes learning by developing a complete, working application. Each chapter introduces new concepts and applies them through practical, hands-on exercises that progressively expand the capabilities of our AI-powered Personal Assistant.

Using ChatGPT for planning and design and Codex for implementation, readers will develop corresponding applications for Android and Windows, Linux, or macOS, combining conversational interfaces, relational databases, document management, and semantic search.

By the end of the book, readers will have developed two synchronized, AI-powered applications with equivalent functionality and gained the practical knowledge and experience needed to apply these technologies to their own software projects.

Learning by Prompting

This book adopts a Learning by Prompting approach, using ChatGPT as an interactive learning companion throughout the development process.

Rather than providing extensive step-by-step instructions for installing software, learning development tools, or executing operating-system commands, the book introduces suggested prompts that readers can submit directly to ChatGPT.

For example, instead of dedicating an entire chapter to Git and GitHub, readers can use the following prompts:

Suggested Prompts

- How do I install Git on Windows, Linux, or macOS?
- How do I create a GitHub account?
- Give me a quick introduction to Git and GitHub.
- How do I enable Git for my Android Studio project?
- Explain the basic Git commands with examples.

ChatGPT can provide instructions tailored to the reader's operating system, development environment, and level of experience. Readers can ask follow-up questions, request clarification, explore alternative approaches, and troubleshoot problems as they arise.

Unlike traditional textbooks, where instructions remain fixed until the next edition, this approach allows readers to obtain guidance that can reflect changes in software versions, development tools, and operating systems.

Learning by Prompting also encourages active participation. Readers are not expected to accept AI-generated instructions blindly. They should understand the purpose of each operation, examine the results, and verify that the suggested solutions work correctly.

This approach allows the book to concentrate on its primary objectives: application architecture, software design, implementation, conversational databases, semantic search, and AI integration, while ChatGPT provides personalized, interactive guidance for supporting development tasks.

The underlying principle is simple: Rather than attempting to teach everything in the book, we teach readers how to use AI to learn what they need, when they need it.

PART ONE

In Part One, we use ChatGPT as a development assistant to build Phone Book, a small Android application demonstrating conversational access to a database of names, addresses, and phone numbers. We prepare the development environment, create the Android Studio project, set up Git and GitHub, and define the application's requirements and architecture. We then begin implementing and improving the interface, exploring screen layouts, Android activities, debugging, and internationalization.

Throughout this work, we learn a repeatable development process: describe a feature, confirm ChatGPT's understanding, refine the requirements, generate and review the code, build and test the application, and save the verified changes. Through this "learning by prompting" approach, we develop practical skills in collaborating with AI while maintaining control over the software, preparing for the more sophisticated Personal Assistant we will develop with Codex in Part Two.

Chapter 1. Prepare the development environment

1.1 Hardware

To develop, test, and run the applications described in this book, you will need the following hardware:

- A computer running Windows 10 or 11, Linux (preferably Ubuntu), or macOS, with an Internet connection, at least 16 GB of RAM (8 GB may be sufficient but could affect performance), and a 256 GB or larger storage drive, preferably an SSD, with 50GB of free space.
- Linux Server: A server running Linux, preferably Ubuntu. The server can be hosted locally or remotely.
- Android Device: An Android phone running Android 8.0 or later. A SIM card or mobile service plan is not required; a Wi-Fi connection is sufficient. An Android tablet can also be used instead of a phone. The phone has to be in Developer Mode. Use this prompt for find out how to enable Developer Mode on your phone.

  - How do I enable Developer Mode on a [your phone]

- USB Cable: A USB data cable to connect the Android device to the development computer for application installation, testing, and debugging.

1.2 Software

1.2.1 ChatGPT

To use ChatGPT, open https://chatgpt.com/ in a browser on your Windows, Linux, or Mac computer. Create an account and start asking questions or giving instructions, called prompts. You can also install the ChatGPT desktop application on Windows, Mac, or supported Linux computers or on your phone. Visit https://chatgpt.com/download/ for installation instructions.

ChatGPT offers free and paid plans. Free usage is limited, and you may have to wait for your usage allowance to reset. A paid subscription provides higher limits and additional features.

For Part One of this book, you need only ChatGPT Plus ($20/month). In Part Two, we will use Codex for AI-assisted programming. Depending on your usage, you may need a paid subscription or additional credits.

Suggested prompts:

- How do I use ChatGPT
- How much does ChatGPT costs
- How do I install ChatGPT desktop on my Windows/Linux/Mac
- Familiarize me with ChatGPT web and desktop interface
- How do I work with projects
- What plugins do I need
- Can I develop Android apps on my Mac machine
- What is Codex
- What is the difference between ChatGPT and Codex
- What is a GPT

Note: Enter these prompts one at a time. Ask follow-up questions whenever you need clarification.

Note: For this book I used GPT-6 and codex-cli 0.166.0-alpha.5.1

1.2.2 OpenAI API

You will need an OpenAI API account. The OpenAI API allows your applications to communicate directly with OpenAI's AI models. You can use it to add conversational interfaces, analyze documents, generate text, and process natural-language commands.

- What is OpenAI API
- How do I install OpenAI API
- How much is OpenAI API costs

1.2.3 Git and GitHub

Git is a version-control system that tracks changes to your source code, allows you to restore previous versions, and supports working on multiple development branches.

GitHub is an online platform for storing, backing up, sharing, and collaborating on Git projects. You will need to create a GitHub account. A free account is sufficient for this book.

- How do I install Git
- How do I access GitHub
- Quick start with Git and GitHub

1.2.4 SSH clients

An SSH (Secure Shell) client allows you to securely connect to a remote Linux server and execute commands as if you were working directly on the server.

Windows, Linux, and macOS include built-in SSH clients. However, one of the most widely used SSH clients is PuTTY, a free, open-source application available for Windows and other platforms.

SSH connections sometimes disconnect because of network interruptions or inactivity timeouts. PuTTY provides a keepalive feature that periodically sends messages to the server to help prevent disconnections caused by inactivity.

- What is Putty
- Does putty work on Linux and Mac
- How to Install PuTTY on Windows
- How to Install PuTTY on Linux
- How to Install PuTTY on Mac
- How do I keep a connection alive in Putty
- How do I use Putty
- How do I use the Windows SSH Client
- How do I use the Linux SSH Client
- How do I use the Mac SSH Client

1.2.5 FileZilla

FileZilla is a free, open-source application used to transfer files between your computer and a remote server. It provides a graphical interface for uploading, downloading, copying, deleting, and managing files.

FileZilla Client is available for Windows, Linux, and macOS. It supports several file transfer protocols, including SFTP (SSH File Transfer Protocol), which allows secure file transfers between your computer and a Linux server.

FileZilla is particularly useful for transferring application files, scripts, and configuration files between your development computer and the server.

- How do I install FileZilla
- Give me a quick start with FileZilla

1.2.6 Android Studio

Android Studio is the official Integrated Development Environment (IDE) for developing Android applications. It is free and available for Windows, Linux, and macOS.

Android Studio provides tools for writing, editing, building, testing, and debugging Android applications. It supports Kotlin, the primary programming language used in this book.

Android Studio includes an Android Emulator, allowing you to test applications without a physical Android device. However, using a real Android phone or tablet is recommended for testing.

- What is Android Studio
- What are the requirements for an Android Studio installation
- How do I install Android Studio
- How do I verify my Android Studio installation
- Give me a short preview on Android Studio

Note: Android Studio install file takes a long time to download.

Note: At the end of Android Studio installation you will be asked to create a project. Create an "No Activity" empty project. Name the project "Phone Book".

Note: Watch for updates. You will get a message box on the lower right corner

Chapter 2. Getting Started

2.1 Phone Book project

If you didn't create the project during installation open Android Studio and click New Project button on the top of the screen. Select No Activity and click next. Set you project name to "Phone Book". Remove example from Package Name. The package name should be com.y219.phonebook. Click Finish. The project will open and on the lower left side of the screen you will see the message Importing gradle Phone Book Project with a progress bar (it will take a while to finish) followed by Analyzing Project. Wait for import and analysis to finish.

Click on the project icon on the upper right of the screen , under the android icon. If you don't see the icon click on the hamburger icon on the upper right corner and select View item, Windows Tools and Project, On the left pane you will see the project window.

Click app→kotlin+java→com.y219.phonebook then expend res, manifest and gradle scripts. You should see the following tree:

These are the most important project items:

- AndroidManifest.xml file – Defines essential information about the application, including its components (activities and services), required permissions, and configuration settings.
- com.y219.phonebook folder – Contains the application's Kotlin source code, including activities, classes, and other program logic. The folder name corresponds to the application's package structure.
- res folder – Contains application resources such as images, icons, colors, text strings, and user interface layouts.
- build.gradle.kts (project) file – Defines project-level build configuration, including plugins and settings shared across modules.
- build.gradle.kts (module) file – Defines the build configuration for a specific module, including application ID, minimum and target Android versions, dependencies, and build options.

- gradle.properties file – Contains Gradle configuration properties that control build behavior, performance, and JVM settings.
- libs.versions.toml file – Defines a centralized catalog of library and plugin versions, allowing dependencies

Go to C:/Users/[your user name] on window, or ~ on Linux and Mac and look for this directory

AndroidStudioProjects \PhoneBook. This is where our project files are stored. Familiarize yourself with the directory structure and files. Compare it with the project structure in Android Studio.

Open PowerShell in Window or terminal in Linux and Mac and go to the project home directory

and type

/PhoneBook > git init

You should get:

Initialized empty Git repository in ….

If you get invalid command install git. You also have to install git 'Command-Line Interface'

- How do I install git and git Command-Line Interface
- How do I create my Phone Book repository on GitHub

Try a few git commands

git status - you will see all the files in the project home directory in red. That means that the files have not been stored in git and they have to be created. Every time a file is update in the project directory structure it will appear in red when you time the command

Notice the file app/src/androidTest/java/com/y219/phonebook/ExampleInstrumentedTest.kt

We do not want to store this file in git as it is unrelated to the project. Open the .gitignore and paste the subdirectory and name of the file in .gitignore. If you try git status again the file will not show in the listing.

git add . (notice the dot after add). This command will stage the file for being committed. The dot tells git to stage all the files. You can stage only one file by replacing the dot with the file name.

git status - will display all the files in green meaning that the files were staged. You can upstage the file using: git restore --staged .

Now we are ready to commit the files, meaning store a new version of the files in git

git commit -m"Initial commit" - the message is mandatory. If you try again git status you will not see any files. This means that the last version of the files in git are identical with the files in the project.

If you created a repository in GitHub you have to connect the local git with the repository in GitHub. You can create and connect with a repository in one step using this command in your project home directory

git remote add origin https://github.com/YOUR_USERNAME/PhoneBook.git

then check if you are connected

git remote -v

YOUR USERNAME is the username you use to log in GitHub

? How do I create a new project in github

? How do I install github

? Give me an introduction to github

? How do I connect local git to github repository

Now push the local commit to the remove directory in GitHub

git branch -M main

git push -u origin main

This is enough to get you going with git and GitHub

Chapter 3. Understanding the project

3.1 What will we build

We identified the high level requirements for this project.

Develop a phone book Android app based on a conversational database.

This is a simple application storing Names, addresses and phone numbers. But what is a conversational database?

A conversational database usually means a database you can ask questions in plain language. The database stores the data, and a chat interface translates your question into a query, often SQL, then presents the results.

We can get more details from ChatGPT.

- Give me a Google Cloud's conversational analytics overview
- What is a Big Query
- Do we need Big Query for our Phone Book project

2.2 Languages, libraries and frameworks

We have to identify the tools to use to develop an Android Conversational Database

- What tools, libraries, languages plug-ins etc. do we have to use to build the phone Book app

From the suggestions we identified the language to use:

- Kotlin - Main programming language for the Android application.
- SQL- Create and query the contacts database.
- Python - Implement the AI server and process requests.
- JSON - Exchange structured data between Android and the server.
- Kotlin DSL Configure Gradle build scripts (build.gradle.kts).
- XML - Define Android resources and configuration, including the application manifest.
- Qt - is a cross-platform framework for building desktop and mobile applications

You should get familiar with these languages:

- Give me a quick start up with Kotlin
- Give me a quick start up with SQL
- Give me a quick start up with Python

We also identified libraries and frameworks:

- Jetpack - Compose Build the Android user interface
- SQLite - Store contact records in a relational database on the phone
- Room - Simplify and organize Android database access
- Retrofit / OkHttp - Communicate with the AI server over HTTP
- Kotlin Coroutines - Execute database and network operations without blocking the user interface
- FastAPI - Provide HTTP endpoints on the Python server
- Uvicorn - Run the FastAPI server
- OpenAI Python SDK - Communicate with OpenAI models from the server

Get help from ChatGPT to familiarize yourself with these packages. You will learn more when we start developing.

2.2.1 Application architecture

The following diagram illustrates the architecture of the Phone Book application, showing its major components, the software technologies used, and how information flows between them.

The application runs entirely on an Android phone, including the user interface, application logic, local AI server, and relational database. The local AI server communicates with the OpenAI API over the Internet to interpret natural-language requests.

The architecture consists of five main components:

1. User Interface (Kotlin and Jetpack Compose): Provides a conversational interface where users enter requests in natural language or SQL. Query results are displayed in a scrollable data grid.
2. Application Logic (Kotlin and Coroutines): Processes user requests, determines whether they contain natural language or SQL, and directs them to the appropriate component. It also coordinates database operations and returns results to the user interface.
3. Local AI Server (Python, FastAPI, Uvicorn, and OpenAI SDK): Runs on the Android phone and handles natural-language requests. It communicates with the OpenAI API to interpret user instructions and generate SQL statements.
4. Database Layer (Room and SQLite): Manages access to the local relational database, executes SQL queries, and retrieves matching contact records. The database contains a single table named contacts.
5. Data Grid (Jetpack Compose): Presents query results in a scrollable table, with each row representing a contact record and each column representing a database field.

2.2.2 How the Application Works

When a user submits a request, the application determines how to process it.

Natural-language requests: The application sends the request to the local AI server, which uses the OpenAI API to generate the corresponding SQL query.

1. SQL statements: The application sends the SQL statement directly to the database layer without requiring AI processing.
2. Query execution: The database layer executes the SQL query against the local contacts table.
3. Results: The retrieved records are returned to the application and displayed in the data grid.

This architecture separates the user interface, application processing, AI interpretation, and database management into distinct components. This separation simplifies development, testing, debugging, and maintenance.

Note: Although the AI server runs locally on the Android phone, the proposed implementation uses the cloud-based OpenAI API. Therefore, natural-language processing requires an Internet connection, while SQL queries can be executed locally without Internet access.

Chapter 3. The development process

We are now ready to begin developing the Phone Book application. At first, the task may seem overwhelming, especially when considering the different programming languages, development tools,

libraries, databases, and AI technologies involved. However, with ChatGPT as our development assistant, we can approach the project systematically, one step at a time. We do not need to master every technology before getting started. Instead, we will learn as we go, using ChatGPT to help us design the application, generate and explain code, configure the development environment, identify and correct errors, and test each component. By breaking the development process into small, manageable tasks, we will gradually transform our requirements into a working application. Along the way, we will demonstrate how AI-assisted development can simplify complex programming tasks, accelerate learning, and make modern application development more accessible, even to programmers unfamiliar with some of the technologies involved.

3.1 AI-Assisted Development Process

We will follow an iterative, step-by-step development process using ChatGPT as our programming assistant. Rather than asking ChatGPT to develop the entire application at once, we will divide the project into small, manageable features. Each feature will go through a structured process of requirements clarification, implementation, code review, and testing before we move on to the next feature.

The development process consists of the following steps:

3.1.1 Step 1. Define the Feature Requirements

Start by describing the feature you want to implement in plain English. Explain what the feature should do, how the user will interact with it, and what results are expected.

You do not need to use formal software engineering terminology. ChatGPT can help transform your description into clear, structured requirements.

3.1.2 Step 2. Ask ChatGPT to Explain Its Understanding

Before requesting any code, ask ChatGPT to restate the requirements as it understands them.

For example:

“Review the requirements and explain your understanding. List the expected functionality, identify any ambiguities, and ask questions if clarification is needed. Do not generate code yet.”

This step is particularly important because ChatGPT may interpret your instructions differently from what you intended.

3.1.3 Step 3. Review and Refine the Requirements

Carefully review ChatGPT's interpretation. If something is incorrect, incomplete, or unclear, provide additional instructions.

Ask ChatGPT to update the requirements and restate them. Repeat this process until you are satisfied that the requirements accurately describe the intended functionality.

This iterative dialogue helps identify misunderstandings before they become programming errors.

3.1.4 Step 4. Ask ChatGPT to Implement the Feature

Once the requirements are clear, ask ChatGPT to implement the feature.

For example:

“Implement the approved requirements in Kotlin. Provide the necessary source code, identify which files need to be created or modified, and explain how to integrate the changes into the existing Android Studio project. Do not introduce functionality beyond the approved requirements.”

ChatGPT may generate new source files or recommend modifications to existing files.

3.1.5 Step 5. Update the Source Code

Apply the changes suggested by ChatGPT to the Android Studio project.

Depending on the feature, this may involve creating files, replacing existing code, modifying functions, or updating project configuration files.

When modifying existing code, provide ChatGPT with the relevant source files so that its recommendations are based on the actual implementation rather than assumptions.

Use Git to track changes and preserve working versions of the application.

3.1.6 Step 6. Review and Understand the Code

Before accepting the implementation, review the generated code.

Ask ChatGPT to explain any functions, statements, libraries, or programming techniques you do not understand.

For example:

“Review the implementation against the approved requirements. Explain how the code works, identify possible problems, and suggest corrections if necessary.”

Remember that AI-generated code is not guaranteed to be correct. ChatGPT can make mistakes, overlook existing functionality, or suggest code that does not compile.

The developer remains responsible for reviewing and verifying the implementation.

3.1.7 Step 7. Build and Debug the Application

Compile the application using Android Studio.

If compilation errors occur, copy the relevant error messages and submit them to ChatGPT for analysis.

For example:

“The following compilation error occurred. Explain the cause and provide the necessary correction.”

Apply the suggested corrections and rebuild the application. Repeat until the code compiles successfully.

A successful compilation confirms that the code can be built, but it does not guarantee that the feature behaves correctly.

3.1.8 Step 8. Request and Execute Unit Tests

Ask ChatGPT to generate unit tests based on the approved requirements.

For example:

- Create JUnit unit tests for this feature. Include normal operating conditions, boundary conditions, and error cases. Explain how to execute the tests in Android Studio.

Add the generated tests to the project and execute them.

If a test fails, provide ChatGPT with the test results and relevant source code. Ask it to determine whether the problem is in the implementation or in the test itself.

Continue correcting and testing until the feature passes its tests.

3.1.9 Step 9. Perform Functional Testing

Run the application on an Android emulator or physical phone and verify that the feature behaves as expected.

Unit tests verify individual components, while functional testing confirms that the feature operates correctly from the user's perspective.

Check that the new feature works as specified and does not interfere with existing functionality.

If unexpected behavior occurs, describe the problem to ChatGPT and provide any relevant Logcat messages.

3.1.10 Step 10. Save and Document the Completed Feature

Once the feature has been implemented and successfully tested:

- Update the project documentation to reflect the implementation.
- Commit the changes to Git with a meaningful description.
- Push the changes to GitHub when appropriate.
- Proceed to the next feature.

3.2 The Iterative Development Cycle

The process follows a repeating cycle:

Define Requirements ? Confirm Understanding ? Refine Requirements ? Implement ? Review Code ? Build ? Test ? Correct ? Commit

Each completed feature becomes part of the growing application. If problems are discovered, we return to the appropriate step, correct the issue, and repeat the necessary tests.

3.3 An Important Principle

Do not ask ChatGPT to implement a requirement until you are satisfied that it understands exactly what you want.

A few minutes spent clarifying requirements can save considerable time debugging incorrect implementations.

Throughout this process, we will treat ChatGPT as a development assistant rather than an infallible programmer. We will use its capabilities to accelerate coding, improve our understanding of unfamiliar technologies, and help diagnose problems, while maintaining control over requirements, design decisions, and acceptance of the final implementation.

This approach demonstrates one of the most important advantages of AI-assisted software development: we can build complex applications incrementally, learning and verifying each step rather than attempting to understand and implement everything at once.

3.4 Prompt Engineering

Prompt engineering is the practice of designing clear and effective instructions, called prompts, to guide artificial intelligence systems such as ChatGPT toward producing useful and accurate results.

In traditional programming, developers write detailed instructions using a programming language. With AI-assisted development, we can describe what we want to accomplish in natural language and ask ChatGPT to help generate the necessary code.

However, the quality of the results depends significantly on the quality of the prompts. Vague or incomplete instructions may produce code that does not meet our expectations, while clear, specific prompts generally lead to more accurate and useful responses.

3.4.1 Writing Effective Prompts

When using ChatGPT for software development, follow these basic guidelines:

1. Be specific. Clearly describe what you want to accomplish, including the expected behavior and any constraints.
2. Provide context. Identify the application, programming language, relevant source files, and existing functionality.
3. Break complex tasks into smaller steps. Ask ChatGPT to implement one feature at a time rather than attempting to develop the entire application in a single prompt.
4. Confirm understanding. Before requesting code, ask ChatGPT to restate the requirements and identify any ambiguities.

5. Specify constraints. Tell ChatGPT what it must not change, particularly when modifying existing code.
6. Review and refine. If the response is incomplete or incorrect, provide feedback and ask ChatGPT to revise it.
7. Request explanations and tests. Ask ChatGPT to explain generated code and provide appropriate unit tests.

3.4.2 Example: Improving a Prompt

Consider the following simple prompt:

- Create a database for my Phone Book application.

Although ChatGPT can respond to this request, it must make assumptions about the database structure and implementation.

A more effective prompt would be:

- We are developing an Android Phone Book application using Kotlin and SQLite. Create a contacts table using the fields defined in our approved requirements. Explain the SQL statements and provide the Kotlin code necessary to create the table. Do not add fields or functionality that have not been specified. Before generating code, summarize your understanding of the requirements.

The second prompt provides context, identifies the technologies, establishes constraints, and requests confirmation before implementation.

3.4.3 Prompt Engineering Is an Iterative Process

An effective prompt does not always produce a perfect response on the first attempt. Prompt engineering involves an ongoing conversation in which we review results, correct misunderstandings, provide additional information, and refine our instructions.

For example, if ChatGPT generates code that modifies unrelated functionality, we can instruct it:

- Revise the implementation to preserve all existing functionality. Modify only the code necessary to implement the approved requirement.

We can also ask ChatGPT to explain its proposed changes before applying them.

3.4.4 Prompt Engineering in Our Project

Throughout the development of the Phone Book application, we will use prompt engineering as a fundamental development technique. We will describe requirements in natural language, ask ChatGPT to confirm its understanding, generate code, review implementations, and create tests.

The objective is not simply to generate code faster, but to produce software that satisfies clearly defined requirements and can be understood, tested, and maintained.

As we progress through the project, we will develop practical prompt engineering skills through hands-on experience. These skills are applicable not only to Android development but also to other programming languages, platforms, and software engineering activities.

Chapter 4. Implementing the frontend

4.1 First compile

Let's begin by compiling the project we created. Click the hamburger icon on the upper left corner to display the main menu. On Mac the menu is always visible. Select Build and then Compile Phone Book app. The compile process starts and on the lower right corner you will see a message Gradle Build Running.

Gradle is the build tool that turns your Android project's code and resources into an installable app, such as an APK. It manages dependencies, runs tests, and handles build settings like Android versions and debug or release builds. Android Studio uses Gradle behind the scenes, with configuration stored in files such as build.gradle.kts.

As a matter of fact there are two build.gradle.kts

build.gradle.kts(project: Phone Book) is the project-level build configuration written in Kotlin. It usually declares build plugins and their versions, such as the Android and Kotlin plugins, for modules in your Phone Book project to use.

// Top-level build file where you can add configuration options common to all sub-projects/modules.

```
plugins {
    alias(libs.plugins.android.application) apply false
}
```

build.gradle.kts(Module: app) configures the app itself, including its Android SDK versions, application ID, and dependencies.

```
plugins {
    alias(libs.plugins.android.application)
}
android {
    namespace = "com.y219.phonebook"
    compileSdk {
        version = release(37)
   }

    defaultConfig {
        applicationId = "com.y219.phonebook"
        minSdk = 24
        targetSdk = 37
        versionCode = 1
        versionName = "1.0"
        testInstrumentationRunner =
```

```
"androidx.test.runner.AndroidJUnitRunner"
    }

    buildTypes {
        release {
           optimization {
                enable = true
                packageScope = setOf("androidx.**", "kotlin.**",
"kotlinx.**")
            }
        }
    }

    compileOptions {
        sourceCompatibility = JavaVersion.VERSION_11
        targetCompatibility = JavaVersion.VERSION_11
    }
}

dependencies {
    implementation(libs.androidx.appcompat)
    implementation(libs.androidx.core.ktx)
    implementation(libs.material)
    testImplementation(libs.junit)
    androidTestImplementation(libs.androidx.espresso.core)
    androidTestImplementation(libs.androidx.junit)
}
```

The build.gradle.kts (Module: app) file usually contains three main sections:

- plugins { } enables tools needed to build the app, such as the Android application and Kotlin plugins.
- android { } configures Android settings. These include namespace, compileSdk, a defaultConfig block with the application ID, minimum Android version, and app version, plus build types such as debug and release.
- dependencies { } lists libraries the app uses, such as AndroidX, Jetpack Compose, and testing tools.

Another important file is the AndroidManifest.xml. The AndroidManifest.xml file tells Android what your app contains and what it needs. Its main parts are <manifest>, the root element; <uses-permission>, which declares permissions such as internet or camera access; and <application>, which defines app settings such as its icon, label, and theme. Inside <application>, you declare components such as activities, services, and broadcast receivers. An activity's <intent-filter> can specify that it is the app's entry screen when launched from its icon.

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android=http://schemas.android.com/apk/res/android
```

```
    xmlns:tools="http://schemas.android.com/tools">
    <application
        android:allowBackup="true"
        android:dataExtractionRules="@xml/data_extraction_rules"
        android:fullBackupContent="@xml/backup_rules"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:roundIcon="@mipmap/ic_launcher_round"
        android:supportsRtl="true"
        android:theme="@style/Theme.PhoneBook" />
</manifest>
```

The build.gradle.kts and AndroidManifest.xml files will change continuously as the app is developed. When build.gradle.kts changes you will not be able to compile until you sync the project with the Gradle files. To so this open the menu and under File scroll down to Sync Project with Gradle Files.

4.2 Creating the Phone Book Interface

To allow users to interact with our Phone Book app, we will create a screen similar to the ChatGPT interface. The screen will include a conversation area, a text field for entering requests, and a send button. Users will be able to type requests such as “Add a new contact,” “Find John’s phone number,” or “Update Mary’s email address.” The app will display responses in the conversation area and ask for any additional details needed to complete each request.

To create this screen, we have to define its layout in an XML file, such as activity_main.xml. We will add a conversation area to display messages, a text field for entering requests, and a send button. In MainActivity.kt, we will load this layout using setContentView() and connect its elements to the Phone Book’s contact management logic.

? Create a well formatted XML file activity_main.xml containing a conversation area to display messages, a text field for entering requests, and a send button and display the file. Add extended comments

ChatGPT should create this file:

```
<?xml version="1.0" encoding="utf-8"?>

<!--

   Main Phone Book screen.

   MainActivity loads this file with setContentView(R.layout.activity_main).

   This XML defines the views; Kotlin code handles sending requests and replies.

-->

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
```

```
android:layout_width="match_parent"
android:layout_height="match_parent"
android:orientation="vertical"
android:padding="16dp">

<!--
    Conversation scrolling container.
    A height of 0dp with weight 1 gives it the space left above the input row.
    fillViewport makes its child at least as tall as the visible area.
    A ScrollView supports exactly one direct child: conversationArea below.
-->
<ScrollView
    android:id="@+id/conversationScroll"
    android:layout_width="match_parent"
    android:layout_height="0dp"
    android:layout_weight="1"
    android:fillViewport="true">

    <!--
        Message container, initially empty.
        MainActivity can append TextViews for user messages and app replies.
        Vertical orientation places each message below the previous one.
        After adding a message, Kotlin code can scroll conversationScroll
        to the bottom so the latest response is visible.
```

```
    -->
    <LinearLayout
        android:id="@+id/conversationArea"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:orientation="vertical"
        android:paddingBottom="16dp" />
</ScrollView>

<!--
    Request composer.
    This row stays below the conversation because it is outside the ScrollView.
    Bottom gravity aligns the button with the bottom of a growing input field.
    Start-based spacing also supports right-to-left interface layouts.
-->
<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:gravity="bottom"
    android:orientation="horizontal">

    <!--
        Request text field.
        Width 0dp and weight 1 let it take the width remaining beside Send.
```

```
    Multiline input supports longer requests, showing up to four lines
    before the text scrolls within the field. The hint comes from strings.xml
    for translation. A 48dp minimum height provides a usable touch target.
    Kotlin reads its text and clears it after accepting a request.
-->
<EditText
    android:id="@+id/requestInput"
    android:layout_width="0dp"
    android:layout_height="wrap_content"
    android:layout_weight="1"
    android:gravity="top|start"
    android:hint="@string/request_hint"
    android:inputType="textCapSentences|textMultiLine"
    android:maxLines="4"
    android:minHeight="48dp" />

<!--
    Send action.
    The visible text also supplies the button's accessibility label.
    MainActivity attaches a click listener, rejects blank input, displays
    the user's message, and passes the request to contact-management logic.
    Declaring this button alone does not implement those behaviors.
-->
<Button
```

```
        android:id="@+id/sendButton"

        android:layout_width="wrap_content"

        android:layout_height="wrap_content"

        android:layout_marginStart="8dp"

        android:minHeight="48dp"

        android:text="@string/send_request" />

    </LinearLayout>

</LinearLayout>
```

Read the comments carefully and familiarize with the format.

Now we add this file to our project. In the project pane expend app then res folders. Right click on layout and under New select Create Resource File. Enter activity_main.xml and click OK. You may be asked to add the file to git. Answer yes. You will see the Layout as it will appear on the phone's screen (currently empty). To see the code click hamburger icon on the right side of the file name.

Remove all the code in the file and copy and paste the code generated by ChatGPT. Click the image icon on the right of the hamburger. You should see the layout of the screen.

We have to create the code to display this layout on the phone's screen. Expand app ? kotlin+java and right-click on the first com.y219.phonebook (nothing in parentheses). Select New ? Kotlin Class/File. Enter MainActivity, select Class, and press Enter. Android Studio creates an empty MainActivity.kt. We will ask ChatGPT to create the code.

- Create the code for MainActivity.kt to handle the layout activity_main.xml we created earlier. Add extensive comments. The package name is com.y219.com.phonebook. Display the code.

Chat GPT should create the following code:

```
package com.y219.phonebook

import android.os.Bundle

import android.view.View
```

```
import android.widget.Button
import android.widget.EditText
import android.widget.LinearLayout
import android.widget.ScrollView
import android.widget.TextView
import androidx.appcompat.app.AppCompatActivity
import androidx.core.view.ViewCompat
import androidx.core.view.WindowInsetsCompat

/** Displays the XML screen and handles the user's Send action. */
class MainActivity : AppCompatActivity() {
    // These references become available after setContentView creates the views.
    private lateinit var requestInput: EditText
    private lateinit var conversationArea: LinearLayout
    private lateinit var conversationScroll: ScrollView
    // Keep messages so they can survive activity recreation.
    // This is temporary screen state, not a permanent contact database.
    private val messages = arrayListOf<String>()

    override fun onCreate(savedInstanceState: Bundle?) {
        super.onCreate(savedInstanceState)
        // Load the XML layout before looking up any of its views.
        setContentView(R.layout.activity_main)
        // Connect Kotlin variables to the IDs defined in activity_main.xml.
```

```
requestInput = findViewById(R.id.requestInput)
conversationArea = findViewById(R.id.conversationArea)
conversationScroll = findViewById(R.id.conversationScroll)
val sendButton = findViewById<Button>(R.id.sendButton)
// Keep content clear of system bars, display cutouts, and the keyboard.
// Save the original XML padding so repeated updates do not accumulate it.
val root = conversationScroll.parent as View
val left = root.paddingLeft
val top = root.paddingTop
val right = root.paddingRight
val bottom = root.paddingBottom
ViewCompat.setOnApplyWindowInsetsListener(root) { view, insets ->
    val safeArea = insets.getInsets(
        WindowInsetsCompat.Type.systemBars() or
            WindowInsetsCompat.Type.displayCutout() or
            WindowInsetsCompat.Type.ime()
    )
    view.setPadding(
        left + safeArea.left, top + safeArea.top,
        right + safeArea.right, bottom + safeArea.bottom
    )
    insets
}
ViewCompat.requestApplyInsets(root)
```

```
    // Restore messages after activity recreation, such as screen rotation.
    // Android restores the input field's state using its stable XML ID.
    savedInstanceState?.getStringArrayList("messages")?.let {
        messages.addAll(it)
    }
    messages.forEach { displayMessage(it) }
    // Run this listener whenever the user taps Send.
    sendButton.setOnClickListener {
        val request = requestInput.text.toString().trim()
        // Ignore empty requests and show an error beside the input field.
        if (request.isBlank()) {
            requestInput.error = "Enter a request first"
            return@setOnClickListener
        }
        addMessage("You: $request")
        requestInput.text.clear()
        // Temporary reply: replace this with contact-management logic.
        // This example does not read, add, or change contacts.
        addMessage("Phone Book: Received your request.")
    }
}

private fun addMessage(message: String) {
    // Save the message, then create the view that displays it.
```

```
    messages.add(message)
    displayMessage(message)
}

private fun displayMessage(message: String) {
    // Create one TextView for each message in the conversation.
    val messageView = TextView(this).apply {
        text = message
        textSize = 16f // Text size uses scalable pixels (sp).
        // Padding requires pixels, so convert 12dp using screen density.
        val spacing = (12 * resources.displayMetrics.density).toInt()
        setPadding(0, spacing, 0, spacing)
    }
    conversationArea.addView(messageView)
    // Wait until Android lays out the new message before scrolling.
    conversationScroll.post {
        conversationScroll.fullScroll(View.FOCUS_DOWN)
    }
}

override fun onSaveInstanceState(outState: Bundle) {
    // Preserve this small conversation when Android recreates the activity.
    // A growing production history should use persistent storage instead.
    outState.putStringArrayList("messages", ArrayList(messages))
```

```
        super.onSaveInstanceState(outState)

    }

}
```

Remove all the code from and paste the code generated by ChatGPT. Read all the comments carefully.

Connect your phone to the computer. You should see the phone name on the top bar:

Click the green bug to compile in debug mode and install on the phone. To see the compiler output click the hummer on the toolbar on the left of the screen, If you do not see the hummer, open the menu, and under View select Tools Windows. On the drop down opened click Build.

Note: If the build.gradle.kts was changed the green bug is disabled.

After compilation ends the app should open on the phone. Click the SEND button, enter some text and SEND. You should see the output of a stab created temporarily until we code the real functionality.

Let's walk though the code in debug mode. First we set breakpoints at the beginning of all functions. Click on the line number on the first line of code in the function. A red bubble will show over the line number. We should have four breakpoints. Click All Breakpoints under the Run menu option. All the breakpoints are listed on the right pane. One the left you can set parameters for the currently selected breakpoint.

Let's add some traces to the log. Add this import is not already included import android.util.Log and add this line as the first line of code in OnCreate function Log.d("MyTrace", "onCreate"). Notice the after typing Log you see a code suggestion. Replace " MainActivity" with "MyTrace" Click tab to accept it. Do this for all functions.

On the lower left side of the screen lick the cat icon. If you don't see it under View?Tool Windows select Logcat. The log window will open. In my case is at the bottom of the screen.

To execute the code click the green bug at the top of the screen. Android Studio will compile the code, install it on your phone and execute it. The execution will start on our first breakpoint. The message we loged will be displayed in the LogCat window.

Click on the bug on the left toolbar. The debug window will open. Hover the cursor over the debug toolbar to familiarize with the buttons. Look for this line

```
val request = requestInput.text.toString().trim()
```

 and right click on . In the context menu select "Add to Watches"

Click Step Over button on the debug toolbar to browse throw the code. Read the comments carefully. When you get to the last statement of OnCreate function click the Resume program button on the debug toolbar. When onCreate() returns, control goes back to the Android framework, which called it.

4.3 Lifecycle of an activity

Add this code inside the class MainActivity, at the end.

```
override fun onStart() {
    super.onStart()
    Log.d("MyTrace", "onStart")
}
override fun onResume() {
    super.onResume()
    Log.d("MyTrace", "onResume")
}
override fun onPause() {
    super.onPause()
    Log.d("MyTrace", "onPause")
}
override fun onStop() {
    super.onStop()
    Log.d("MyTrace", "onStop")
}
override fun onRestart() {
    super.onRestart()
```

```
    Log.d("MyTrace", "onRestart")
}
override fun onDestroy() {
    Log.d("MyTrace", "onDestroy")
    super.onDestroy()
}
```

These callbacks describe an Activity's lifecycle—usually one screen in your app.

Callback	What happens
onCreate()	The activity is created. Set up the screen and initial state.
onStart()	The activity becomes visible.
onResume()	The activity is in the foreground and ready for interaction.
onPause()	The activity loses focus. It might still be visible.
onStop()	The activity is no longer visible.
onRestart()	A stopped activity is about to become visible again.
onDestroy()	The activity instance is being destroyed.

A typical journey is:

Open the screen:

onCreate → onStart → onResume

Leave for another app:

onPause → onStop

Return to the screen, if the activity is still in memory:

onRestart → onStart → onResume

Finish the screen:

onPause → onStop → onDestroy

These aren't always one continuous sequence. An activity can pause and resume many times. Android can also kill your app's process without calling onDestroy(), so don't rely on it to save important data.

To observe the lifecycle, run the app in Debug mode and filter Logcat using tag:MyTrace. Trace through the callback functions and watch their messages as you open the app, press Home, and return to it. Each callback returns control to the Android framework when it ends.

By default, rotating the phone changes the device configuration. Android destroys the current activity and creates a new instance so it can load the layout and resources appropriate for the new orientation. For a fully resumed activity, you will normally see:

Old activity:

onPause

onStop

onDestroy

New activity:

onCreate

onStart

onResume

onCreate() runs again because Android is initializing a new activity instance. This does not mean the entire app process has restarted. You do not see onRestart() in this sequence because the old activity was destroyed rather than reused.

If you also log onSaveInstanceState(), you should see that callback during recreation, although its position relative to other callbacks can vary. In your MainActivity, it saves the messages list into a Bundle, and the new activity restores that list in onCreate(). Ordinary member variables are otherwise initialized again, so state must be preserved deliberately—for example, through saved instance state or a ViewModel.

This rotation sequence assumes the activity uses Android's default configuration handling. An activity configured to handle orientation changes itself may behave differently.

Remove all the breakpoints in the MainActivity code. Under Run select View Breakpoints and click on the checkboxes to disable the breakpoints.

Click the green Debug button (the bug icon) at the top of Android Studio to build, install, and run the application in Debug mode. Interact with the app: press Home, switch to another app, return, rotate the phone, and close and reopen the activity. Watch the lifecycle messages in Logcat using the filter tag:MyTrace, and compare the callback sequence for each action.

Click the terminal icon on the lower right of the screen or open the terminal window using the menu. View ?Tool Windows and scroll down to the Terminal option.

Enter

git status

You should see the files newly created. Stage them, commit and push to GitHub

Article I. git add .

git commit -m"Implemented the screen layout"

git push -u origin main

4.4 Internationalization

Internationalization prepares an application for different languages and regional conventions. Localization supplies the translations and adaptations for a particular language or region. In Phone Book, we begin by separating the text displayed to users from the code that controls the application.

The project already uses this approach for the application name, request hint, and Send button. The default English text is defined in the resource file below.
[strings.xml](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/res/values/strings.xml)

```
<resources>

    <string name="app_name">Phone Book</string>

    <string name="request_hint">Enter a request</string>

    <string name="send_request">Send</string>

</resources>
```

Each string has a resource name that remains consistent across translations. For example, send_request identifies the button label, while Send is its English value.

The screen layout refers to these resource names. The request field uses @string/request_hint, and the button uses @string/send_request. This allows Android to select the appropriate text without changing the layout. [activity_main.xml](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/res/layout/activity_main.xml)

android:hint="@string/request_hint"

android:text="@string/send_request"

To add French, we would create app/src/main/res/values-fr/strings.xml with the same resource names and translated values.

<resources>

    <string name="app_name">Répertoire téléphonique</string>

    <string name="request_hint">Saisissez une demande</string>

    <string name="send_request">Envoyer</string>

</resources>

Android selects resources according to the active language configuration. When an appropriate translation is unavailable, it uses the default resources. The default values/strings.xml file should therefore contain every string the application needs.

Some text in Phone Book still needs to be moved into resources. MainActivity.kt currently contains these English strings directly in the Kotlin code. [MainActivity.kt](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/java/com/y219/phonebook/MainActivity.kt)

requestInput.error = "Enter a request first"

addMessage("You: $request")

addMessage("Phone Book: Received your request.")

To make them translatable, we would add the following entries to the default string resource file.

<string name="request_required">Enter a request first</string>

<string name="user_message">You: %1$s</string>

<string name="request_received">Phone Book: Received your request.</string>

The Kotlin code would then retrieve the text through getString().

requestInput.error = getString(R.string.request_required)

addMessage(getString(R.string.user_message, request))

addMessage(getString(R.string.request_received))

The placeholder %1$s represents the user's request. Keeping the complete message in one resource gives translators control over its wording and the position of the inserted text.

Internationalization also affects layout. Phone Book already declares android:supportsRtl="true" in its manifest and uses direction-aware attributes such as layout_marginStart and gravity="top|start" in its layout. These provide a foundation for languages written from right to left. Translated text may also occupy more space, so the screen should be checked for clipped labels, crowded controls, and alignment problems.
[AndroidManifest.xml](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/AndroidManifest.xml) •
[activity_main.xml](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/res/layout/activity_main.xml)

For this feature, testing would include switching between English and French, checking the input hint and button label, triggering the empty-request error, and sending a request. The current activity saves conversation messages as already formatted strings, so existing messages retain their original wording after a language change. This behavior should be considered when defining the language-switching requirement.

Phone Book currently contains only the default English resources. The French translations and Kotlin changes above illustrate the next implementation step.

4.5 Conclusion

In this chapter, we learned how to create an Android activity and build the app's first screen, giving it the look and feel we intended.

We followed the development lifecycle for a single feature, from defining and refining the requirement to approving it, designing the solution, implementing it, testing the result, reviewing it, and committing the changes.

Along the way, we practiced using ChatGPT to clarify requirements and develop Kotlin code, and Android Studio to build and test the application. This established a foundation for adding features incrementally, with a clear requirement and a verified result at each step.

Chapter 5. Improving the frontend

5.1 Adding an app exit facility

The requirement to add an Exit button is the following:

Add the text Exit on the upper right corner on the blue ribbon. When the use tops the text the app should terminate.

Sent it to ChatGPT

- How do you understand this requirement: Add the text Exit on the upper right corner on the blue ribbon. When the use taps the text the app should terminate.

If you are happy with the answer submit

- Implement. Show me the changes I have to make to the code.

ChatGPT generates the following code:

1. Created [main_menu.xml](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/res/menu/main_menu.xml) to display Exit at the right of the ribbon:

```
<?xml version="1.0" encoding="utf-8"?>

<menu xmlns:android="http://schemas.android.com/apk/res/android"

    xmlns:app="http://schemas.android.com/apk/res-auto">

    <item

        android:id="@+id/action_exit"

        android:title="@string/exit"

        app:showAsAction="always" />

</menu>
```

2. Added to [strings.xml](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/res/values/strings.xml):

```
<string name="exit">Exit</string>
```

3. Added these imports and methods to [MainActivity.kt](C:/Users/gdobr/AndroidStudioProjects/PhoneBook/app/src/main/java/com/y219/phonebook/MainActivity.kt):

```
import android.view.Menu

import android.view.MenuItem
```

Inside the MainActivity class:

```
override fun onCreateOptionsMenu(menu: Menu): Boolean {

    menuInflater.inflate(R.menu.main_menu, menu)

    return true

}



override fun onOptionsItemSelected(item: MenuItem): Boolean {

    return when (item.itemId) {

        R.id.action_exit -> {

            finishAndRemoveTask()

            true

        }

        else -> super.onOptionsItemSelected(item)

    }

}
```

You can ask for clarifications:

- Where exactly should I insert the new functions. Can you add more detailed comments

Review the code generated and understand its functionality. If you do not understand something ask for help.

- What is this function doing onOptionsItemSelected

When you implemented the code and understand the functionality compile the code clicking the green bug on the top menu. The app will be installed on your connected phone.

To test, open first phone Settings - Apps - Phone Book. The option Force Stop should be enabled. Do not force stop the app. Go back to the app screen and tap the Exit text. The app should be terminated. Go back to Settings - Apps - Phone Book.TheForce Stop should be disabled because the app is not running anymore. Set breakpoints on the new functions and run the test again. Check the logs on CatLog.

Finish this cycle by staffing, committing and pushing the code.

5.2 Add an icon and splash screen

Finish this cycle by staffing, committing and pushing the code.

5.3 Add help

5.4 HitHub

5.5 Conclustion

Part Two - Developing a Personal Assistant with Codex

In Part I, we explored how ChatGPT can assist with software development, from defining requirements and designing databases to planning application architecture. In Part II, we move from planning to practical implementation using Codex, OpenAI's AI-powered software development agent. We will develop two corresponding versions of our AI-powered Personal Assistant: an Android application and a desktop application running on Windows, Linux, or macOS, both providing equivalent functionality.

Using prompt-driven development, we will guide Codex through creating, modifying, debugging, and testing the applications. Each application will maintain its own local relational database and document library, synchronized across devices, and support conversational interaction, database operations, and semantic search. Throughout the process, we will review and test the generated code, demonstrating how programmers can collaborate effectively with AI to develop complete, working applications.

Chapter 6. Using ChatGPT for Software Market Research

Before developing a new software application, it is important to understand the market, identify potential users, evaluate competing products, and determine whether there is a genuine need for the proposed application.

Traditionally, market research required considerable time searching websites, reading product reviews, participating in discussion forums, and comparing competing applications. ChatGPT can significantly simplify and accelerate this process.

6.1 Researching the Market with ChatGPT

ChatGPT can search the internet, examine competing applications, summarize customer feedback, and identify common complaints and missing features. It can also help evaluate pricing models, target audiences, and opportunities for product differentiation.

One particularly valuable source of information is online discussion forums, where users share their experiences, frustrations, and suggestions. These discussions can reveal practical requirements that may not be obvious from competitors' product descriptions.

For our Personal Assistant project, we will investigate existing applications for managing contacts, appointments, reminders, notes, documents, and other personal information across Android and desktop platforms.

- Research the market for personal information management applications running on Android, Windows, Linux, and macOS. Identify leading products, their major features, pricing, customer feedback, common complaints, and missing functionality. Search relevant user forums and software reviews. Include references to the original sources.

ChatGPT can organize the findings into a market research report.

6.2 Analyzing Competing Applications

Once potential competitors have been identified, ChatGPT can help compare their capabilities and identify opportunities for improvement.

Our competition consists primarily of personal information management and productivity applications, rather than general-purpose AI platforms or frameworks.

- Compare leading personal information management applications for Android and desktop computers. Create a table showing their major features, supported platforms, pricing, advantages, disadvantages, and customer feedback. Identify opportunities to differentiate our Personal Assistant.

The results can help determine which features are essential, which are optional, and where improvements might provide a competitive advantage.

6.3 Turning Market Research into Requirements

Market research becomes especially useful when its findings influence application design.

ChatGPT can help translate user complaints and feature requests into proposed software requirements.

? Based on our market research, identify the most important requirements for the Personal Assistant application. Separate essential features from optional enhancements, explain their benefits, and suggest development priorities.

These recommendations should be reviewed before being incorporated into the application's formal requirements. Not every requested feature is practical, and unnecessary functionality can increase development costs and complexity.

6.4 Understanding the Limitations

Although ChatGPT is a powerful research assistant, its findings should not be accepted without verification. Online reviews may be outdated, individual opinions may not represent the broader market, and product features and prices can change.

Ask ChatGPT to provide links to its sources, distinguish verified facts from interpretations, and identify information that could not be confirmed.

Most importantly, market research conducted with ChatGPT does not replace direct feedback from potential customers. Interviews, demonstrations, prototypes, and user testing remain valuable ways to validate demand.

6.5 Conclusion

ChatGPT can transform market research from a lengthy manual process into an interactive investigation. Developers can explore competing products, analyze user experiences, identify unmet needs, and refine their application concepts through natural-language prompts.

By conducting market research early in the development process, programmers can make better design decisions, prioritize useful features, and reduce the risk of building software that does not address real user needs.

The objective is not simply to develop an application that works, but to develop one that people actually want to use.

Chapter 7. Prepare the development environment

7.1 Visual Studio Code

Visual Studio Code (VS Code) is a free, open-source-based code editor developed by Microsoft. It is available for Windows, Linux, and macOS.

VS Code supports many programming languages, including Python, JavaScript, Kotlin, and SQL. It provides features such as syntax highlighting, code completion, debugging, and integrated Git support.

Additional functionality can be installed through extensions, including tools for working with remote Linux servers using SSH.

In this book, VS Code can be used to develop and maintain server-side applications, edit configuration files, and manage source code.

? What is VS Code

? What are the requirements for an VS Code installation

? How do I install VS Code

? How do I verify my VS Code installation

? Give me a short preview on VS Code

Note: Codex may occasionally suggest additional prompts or follow-up actions. While these suggestions can be helpful, they may not always align with your objectives, priorities, or intended development approach. Review them carefully before accepting them. You are in control of the development process, not Codex.

Appendix A. Software Design Document (SDD)

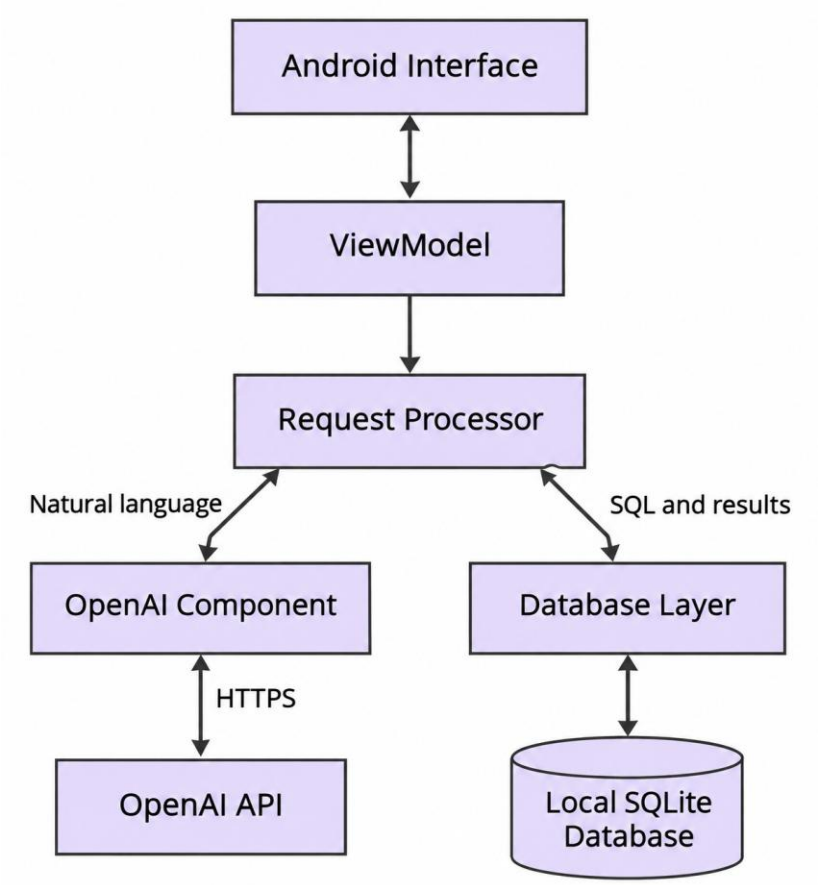
Phone Book Android Application

1. Design Overview

Phone Book is a native Android application developed in Android Studio using Kotlin. It uses XML layouts and Android Views for presentation, a local SQLite database for storage, and an internal Kotlin component for OpenAI API calls.

No separate server runs on the phone.

2. Architecture



3. Components

Component	Design responsibility
MainActivity and	Render conversation messages, request controls,

Component	Design responsibility
XML layouts	query results, and application-bar actions
ViewModel	Hold screen state and coordinate asynchronous processing across activity recreation
Request Processor	Distinguish SQL from natural language, maintain clarification context, validate proposed operations, and coordinate execution
OpenAI Component	Construct HTTPS requests, parse structured responses, and return proposed operations or clarification questions
Database Layer	Manage SQLite through Room, execute permitted operations, and return rows or affected-record counts

The OpenAI component does not execute database operations.

4. Processing Design

Direct SQL passes from the request processor to the database layer after validation.

Natural-language input passes through the OpenAI component. Its response contains a proposed database operation or a clarification question. Proposed operations are validated locally before execution.

Both paths use the same database execution rules. The interface displays results derived from actual execution.

5. Data and State

The local database contains a contacts table. Its physical schema will be finalized from the SRS.

The ViewModel holds conversation messages, query results, processing status, and pending clarification context. Contact records persist in SQLite; permanent conversation storage is not part of this design.

Coroutines keep network and database work off the main thread. Database transactions protect operations requiring atomic changes.

6. Implementation Decisions

- Networking: OkHttp over HTTPS.

- Database access: Room, with a controlled SQL execution boundary.

- Presentation: XML layouts, Android Views, and AppCompat.

- Localization: Android string resources and direction-aware layouts.

- Credentials: Excluded from source control and logs; provisioning remains to be decided.

- Testing: Substitute OpenAI responses for repeatable processing tests; verify database and interface integration in Android Studio.

7. Open Design Decisions

- SQL recognition, permitted statements, and dynamic SQL integration with Room.

- OpenAI request and response format.

- API credential provisioning and storage.

- Results-grid implementation.

Appendix B. Software Requirements Specification (SRS)